

Two Threads Beat One, and the Thread Hop Is a Wash Under Hawkes Arrivals: A Closed-Form and Empirical Case for Multi-Actor HFT Pipelines

Vincent Mayeski*

Draft v2, September 2026

Abstract

The received wisdom in high-frequency-trading (HFT) system design is that the entire hot path — SBE decode, order-book application, strategy touch, order construction — should live on a single-thread event loop, because every additional thread hop adds real wall-clock latency. This paper shows that received wisdom is wrong on real CME Market Data Platform version 3 (MDP3) packet flow, and reports two contrarian conclusions.

First: the thread-hop cost is a wash. The received single-thread wisdom assumes that a $\sim 1.7 \mu\text{s}$ one-way async mailbox hop (Mayeski [31] Table 4; the `fast_send` paper’s $3.4 \mu\text{s}$ round-trip halved) is a material addition to end-to-end latency. On real Hawkes-clustered CME MDP3 packet arrivals, the observed cluster-wait tail is one to two orders of magnitude larger than that per-hop cost across every regime we test, so the hop is not a first-order effect. The single-thread objection — “every hop is a fixed tax that no benefit can overcome” — is empirically wrong on this class of feed.

Second: two threads are strictly better than one. Splitting a service task into two half-length actors on separate threads joined by one $1.7 \mu\text{s}$ mailbox hop is a Pareto improvement on every 30-minute window we test at every service floor $T \gtrsim 2h$. The exact numerical corpus medians come from the $T \in \{2, 4, 8, 16, 32\} \mu\text{s}$ sweep now in flight; the tables in Section 4 will carry the final numbers. The same pattern generalises to an N -stage series tandem with an $N^{-1/2}$ -or-stronger tail-ratio contraction at a linear-in- N median cost, capped by the number of natural cut points in the decode chain (four or five on CME MDP3).

The effect is entirely arrival-law-driven: under a rate-matched Poisson null on the same arrival count, both wins vanish and the multi-actor architecture strictly loses. Two closed-form results anchor the mechanism (a burst-limit throughput identity and a batch-arrival tail-ratio result via the Poisson-cluster representation of Hawkes); a fork-based literature search over foundational tandem theory, Hawkes-adjacent queueing, and empirical HFT/RPC work found no direct prior art on either the theoretical statement or the empirical calibration. Companion code contains the message-tape parser, the per-window packet-Hawkes fitter, and the tandem Lindley simulator.

*M2 Tech (16425640 Canada Inc.), Montreal. v@m2te.ch

Contents

1	Introduction	4
1.1	The design choice, and the safe default	4
1.2	Prior recommendations for single-thread hot paths	4
1.3	What this paper does	6
1.4	Contribution and prior-art positioning	7
1.5	Note on the use of AI in this paper	7
1.6	Notation	7
I	Analytical framework	9
2	Analytical framework	9
2.1	Burst-limit throughput theorem	9
2.2	Cluster representation and tail-ratio theorem	10
2.3	Summary of the analytical predictions	12
2.4	Scope of the results	12
II	Empirical experiment	13
3	Experiment setup	13
3.1	Data corpus	14
3.2	Packet arrivals are Hawkes-clustered (empirical prerequisite)	14
3.3	Simulator	15
3.4	Pipelining is what makes multi-stage work	15
3.5	Sweep design	16
4	Empirical results	16
4.1	Tail collapse under Hawkes, not under Poisson	17
4.2	Median cost is linear in N , exactly as $(N - 1)h$	17
4.3	Design equation	17
4.4	Comparison scenario: $T = 20 \mu s$	18
4.5	Sensitivity to h	19
III	Evaluation and implementation	20
5	Evaluation: do empirics match the analytical predictions?	20
6	Implementing a multi-actor pipeline	20
7	Fungibility of sync and async delivery in KASPAR-HFT	22
7.1	Receiver transparency	22
7.2	Why measurement, not local reasoning, must decide	22
7.3	Operational discipline	23
7.4	Separation of code construction from latency optimization	23
7.5	Contrast with non-fungible frameworks	24

8	Conclusion	24
9	Future work	26
10	Reproducibility	26
A	Structured proof sketch of Conjecture 2	29
A.1	Setting and objective	30
A.2	Step-by-step scaffold	30
A.3	Summary of gaps	31
B	Reading list for a formal proof attempt	32

1 Introduction

1.1 The design choice, and the safe default

The design of the ingress-to-order pipeline in a high-frequency trading system is a choice between two archetypes: the single-threaded event loop that processes every step of every message on one core (SBE decode, book application, strategy evaluation, order construction, all under one call chain, taking a total service time we denote T), and the multi-actor pipeline in which those steps are split across N separate actors, each on its own thread with its own mailbox, connected by $N - 1$ explicit hops whose per-hop wall-clock cost we denote h . It is not really a debate: almost everyone in the low-latency HFT community is in the single-thread camp, and single-threaded is generally viewed as the safe choice. The argument for it cites the cost of a mailbox hop — typically a mailbox enqueue plus a cache-line transfer between the sender’s core and the receiver’s core, plus the cost of the receiver being scheduled or spinning — and holds that $(N - 1)h$ adds directly to every message’s end-to-end latency, so every extra stage in the pipeline is a fixed tax that no throughput or tail benefit can overcome.

That argument implicitly assumes an arrival process for which single-server throughput $1/T$ is comfortably larger than the peak arrival rate. On CME MDP3 the arrival process is strongly self-exciting: packets arrive in Hawkes-clustered bursts [22–24]. During such bursts, the single-thread event loop’s utilisation transiently exceeds 1, and cluster-induced queueing accumulates a tail that dwarfs h by two orders of magnitude. This paper’s central empirical observation is that on 281 sessions of NQ front-month packet arrivals, this cluster tail is what dominates observed end-to-end latency: the 99th-percentile latency (p_{99}) of a $T = 10\mu\text{s}$ constant-service queue driven by real packet arrivals reaches into the hundreds of microseconds during high-clustering windows, orders of magnitude above the service floor. Under a rate-matched Poisson null of the same arrival count the p_{99} sits within a factor of ~ 1.1 of the median; the tail is entirely a Hawkes-cluster effect.

Given that the tail is cluster-driven, the natural question is whether splitting the pipeline into $N > 1$ stages — each running as an independent actor on its own core, with per-stage service T/N , so that the tail-driving cluster-wait is now split across N smaller queues — reduces the tail. If it does, and if the additional median cost $(N - 1)h$ is small relative to the tail savings, the multi-actor architecture is a Pareto improvement over the single-thread design, and the single-thread default is displaced for HFT workloads specifically. All symbols in this paragraph are gathered in the notation table of Section 1.6.

1.2 Prior recommendations for single-thread hot paths

The single-thread rule has a specific lineage in the low-latency-systems community, and we name that lineage here because the paper is arguing against it directly. In reverse chronological order, the lineage has three layers: the HFT-specific codification of the last decade, the general low-latency-systems and event-driven-server culture from which HFT inherited it, and one prominent published counter-argument that the field discarded.

Modern HFT-community codification (2010–present). The founding statement of the current HFT-community formulation is Thompson’s Single Writer Principle [32] — “a single-writer design consistently outperforms a ‘properly’ lock-free or wait-free multi-writer design on most realistic workloads” — motivated by and applied in the LMAX Exchange architecture [33, 34]. Rigtorp’s widely cited low-latency tuning guide [35] formalises the operational rule (“run a single thread in `SCHED_OTHER` per core and use busy waiting/polling”), and Chronicle Software’s public engineering

materials [36] adopt the same architecture for market-data connectors (“within a single event loop ... busy-spinning and affinity locks”), pinning Chronicle Queue’s design to a single writer per queue. The Aeron messaging library [37] inherits the same lineage. The academic side is thinner, but Kwan et al. [38] codify the pattern as the standard low-latency-HFT design pattern, and recent practitioner posts [39] restate it explicitly (“all critical trading logic runs in a single-threaded event loop, eliminating context switches and lock contention entirely”). Inside HFT specifically, the same recommendation has been reinforced by two decades of private prop-trading arms-race practice among the Chicago desks (Getco, Jump, Optiver, DRW; late 2000s), reported second-hand in industry retrospectives but without an open-literature reference.

Older event-driven-server lineage (1995–2010). HFT did not invent this rule; it inherited it from the event-driven-server culture. The single-threaded event loop as an architectural pattern dates at least to Schmidt’s Reactor pattern [40] in the mid-1990s. It was sharpened by Kegel’s C10K essay [41], which named the OS thread-per-connection model as the bottleneck of scalable network servers and argued for event-driven designs. The pattern went mainstream in production with Sysoev’s nginx web server [42], whose single-threaded worker process is explicitly justified as context-switch avoidance, and later with Dahl’s Node.js runtime [43], which pushed the single-threaded event loop into ordinary application code.

Pro-pipelining minority — outside HFT. A legitimate multi-decade pro-pipelining tradition exists in the datacenter and networking-systems literature; it has been the field’s non-single-thread minority throughout the period covered by the previous two paragraphs. The earliest prominent published argument is the Staged Event-Driven Architecture (SEDA) of Welsh et al. [44], which advocated exactly the multi-stage pipeline of independently scheduled event handlers that the present paper’s tandem design revives. Welsh’s 2010 retrospective [45] is often cited as a walk-back to single-thread, but it is actually a hedged pro-pipelining recommendation: it advocates decoupling stages from thread pools, grouping multiple stages within a single thread-pool domain where latency is critical, and putting a separate thread pool and queue in front of any stage with long or nondeterministic runtime — effectively the present paper’s design rule in different vocabulary. Beyond SEDA the pro-pipelining tradition includes the Click Modular Router [46] for staged packet processing; Naiad [47] for low-latency dataflow; Snap [48] for Google’s multi-engine network stack; and the datacenter microsecond-tail-latency scheduler line — Shinjuku [49], Shenango [50], Caladan [51], Perséphone [52] — which argues for centralised multi-core dispatch specifically to control the microsecond-scale tail. Perséphone in particular is targeted at “wide service-time distributions,” the closest existing framing to our arrival-law claim; but the datacenter line frames the problem as heavy-tailed *service*, not clustered *arrivals*. DPDK explicitly supports both run-to-completion and pipeline modes and its documentation recommends pipeline mode “if some set of packets require longer to process” [53]. Inside HFT specifically the pro-pipelining position is rare in the open literature; an obscure Chinese patent [54] describing a pipelined FAST market-data decoder is a lonely commercial artefact of it.

None of the pro-pipelining sources above ties the design recommendation to Hawkes / self-exciting / near-critical arrival dynamics. The closest theoretical hooks are the MMPP-input tandem work of Kim and Chydzinski [14] already cited in Section 1.4 and the batch-arrival tandem phase-transition analysis of Vinnikov [55]. The present paper’s arrival-law-specific mechanism — that the recommendation flips when arrivals are near-critically self-exciting — remains uncontested by prior work.

Yours truly gets a footnote on that list, but with an important caveat.¹ An earlier paper by the present author [31] did not argue for the single-thread principle in its own right. That paper is defensive rather than prescriptive: the actor model had long been dismissed as unsuitable for HFT because of asynchronous-dispatch overhead, and **fast_send** is presented there as the mechanism that eliminates the overhead for co-located actors and thereby makes the actor model usable *inside* a servicing chain that the received wisdom of the time had already placed on a single thread. The reference architecture in that paper — reader $\rightarrow$ buffer via asynchronous **send**, decode/book/signal chain via **fast_send** on one thread, signal $\rightarrow$ order-out via asynchronous **send** — takes the collapsed single-thread servicing chain as the widely accepted design context of the time and delivers a way to compose it out of actors at zero context-switch cost. It does not compare single-thread against more-asynchronous topologies and pick single-thread; it takes the topology as given. The present paper is a retraction of that design context, not of the **fast_send** mechanism: on Hawkes-clustered feeds, the aggregate servicing chain should be split into asynchronous stages once its total service time becomes long compared to the Hawkes-cluster tail. **fast_send** remains the right mechanism for chaining actors that have been placed together on one thread; what changes is where the thread boundaries inside the servicing chain should sit. The corrected design rule is stated in Section 8.

1.3 What this paper does

We answer that question along three axes.

- **Theory.** A closed-form theorem and a closed-form conjecture (Section 2) predict the pipeline behaviour. Theorem 1 shows that under any arrival process an N -stage tandem enjoys an N -factor throughput speedup on isolated bursts. Conjecture 2 composes this with the Hawkes cluster representation to predict an $N^{-1/2}$ power-law tail-ratio compression under Hawkes arrivals in a batch-arrival $M^{[K]}/D/1$ approximation. Corollary 3 shows that under a rate-matched Poisson null no such compression exists.
- **Simulation.** We build an N -stage tandem Lindley recursion on real CME MDP3 packet-arrival streams at a hop cost $h = 1.7\mu\text{s}$ (the one-way async mailbox cost derived by halving the round-trip figure in Table 4 of Mayeski [31], and consistent with Mayeski [30]) and a service floor swept across $T \in \{2, 4, 8, 16, 32\}\mu\text{s}$ to cover the regime from tasks below the hop cost (where no split helps) through tasks well above it (where the tail-win compounds). We sweep $N \in \{1, 2, 4, 8\}$ per scenario on each 30-minute RTH window of the 281-session corpus. Each window is also simulated under a uniform-shuffle Poisson null of the same arrival count as an empirical counterfactual (Section 3.5).
- **Design equation for practitioners.** From the empirical result we extract a specific stage-count rule as a function of the operating $(\bar{\lambda}_{\text{pkt}}, n_{\text{pkt}})$ point, and give a worked configuration for a POV-style algorithm on KASPAR-HFT that hits the recommended N (Section 6).

¹A related author’s-aside: the tail compression reported by this paper had in fact been observed empirically in production well before its mechanism was understood. Splitting a long task into shorter asynchronously connected stages was known to help the tail on a per-strategy basis in the author’s own operational experience, and the working assumption at the time was that the single-thread camp must simply be writing faster inner-loop code — not that the architectural recommendation was itself arrival-law-dependent. The present paper is the retrospective explanation of that anecdote.

1.4 Contribution and prior-art positioning

Two rounds of fork-based literature search covering foundational tandem theory (Jackson, Kelly, Burke, Kleinrock, Whitt), tandem theory under bursty arrivals (Foss–Korshunov, Baccelli–Foss–Lelarge, Heindl, Kim–Chydzinski, Dębicki–Mandjes), Hawkes single-queue and adjacent (Daw–Pender, Chen–Blanchet, Koops–Boxma–Mandjes, Sheldon et al., Gao–Zhu, Bo Li, and 2023–2026 follow-ons), and empirical/applied HFT and RPC-pipeline work (ABIDES, ABIDES-MARL, JAX-LOB, Aquilina–Budish–O’Neill, Dean–Barroso, Parsimon, and the datacenter tail-latency crowd) confirmed that there is no direct prior art on either (i) the theoretical claim that the tandem-vs-single-queue architecture recommendation flips sign as a function of arrival law (Poisson vs Hawkes) or (ii) an empirical CME-calibrated tandem sweep with a Poisson null and measured HFT-framework hop cost. The full reference list appears in Section 10.

1.5 Note on the use of AI in this paper

This paper was written with Claude Code (an agentic instance of Anthropic’s Claude LLM) as a co-author, and the use is disclosed here explicitly. In addition to agentic code generation — writing, editing, and running the message-tape parser, the packet-Hawkes fitter, and the tandem-Lindley simulator on the 281-session corpus, all under the author’s direction — Claude Code was used to build the analytical framework of Part I: it drafted Theorem 1 (an elementary induction), Conjecture 2 (the Hawkes-tandem tail-ratio statement), Corollary 3, and the structured proof scaffold and reading list in Appendix A–B. The plausibility argument for Conjecture 2 was likewise drafted by the model and then reviewed and edited by the author.

A rigorous proof of Conjecture 2 has not been run yet in this paper; it is deliberately deferred to a technical follow-on paper, whose structure and effort estimate are in Appendix A. The plan for that follow-on is to use Claude Code (or a comparable agent) to draft the full proof by composing the named literature building blocks — Baccelli–Foss–Lelarge monotone-separable networks, Asmussen–Foss single-big-batch, Hawkes–Oakes cluster representation, and the near-critical joint scaling limit — and then to review and revise the draft with a heavy-tail queueing theorist. The author’s view is that Claude Code is a capable drafter of composition-of-published-results proofs and is unreliable on the piece that requires genuinely new mathematical creativity (the near-critical scaling limit); the follow-on paper will report empirically on that division of labour.

Beyond the theorems and proof scaffold, Claude Code was used to draft prose across the paper, to run and iterate on the empirical experiment, to conduct the two rounds of prior-art literature search that grounded Section 1.4, and to produce the diagrams, tables, and figures. Every claim, number, and citation was reviewed by the author; any residual errors are the author’s.

1.6 Notation

The following symbols are used throughout.

Symbol	Meaning
T	total service time per message on a single-stage pipeline, in μs
N	number of stages in the tandem pipeline (one actor per stage)
T/N	per-stage deterministic service time in the N -stage tandem
h	per-hop wall-clock cost of the async mailbox handoff between adjacent stages
k	size of an arrival burst (number of packets in one cluster)
$\tau_k^{(N)}$	exit time of the last message of a k -burst under N stages
$W^{(N)}$	steady-state waiting time (sojourn minus service) of a typical message under N stages
p_q	q -th percentile of the end-to-end latency distribution (p_{50} , p_{99} , etc.)
μ, α, β	Hawkes background rate, jump size, and decay rate (§3.2)
$n = \alpha/\beta$	Hawkes branching ratio
K	Borel-distributed offspring cluster size (§2.2)
$\bar{\lambda}_{\text{pkt}}$	mean packet-arrival intensity per 30-min window, packets per second
n_{pkt}	branching ratio of the packet-arrival Hawkes fit per window
$n_{\text{msg}}, n_{\text{pkt}}^{\#}$	per-window message and packet counts
ρ	utilisation, $\rho = \bar{\lambda}_{\text{pkt}} \cdot T$
$F(T)$	Fano factor at window length T : $\text{Var}[C_T]/\text{E}[C_T]$
H	Hurst exponent, from log-log slope of $F(T)$

Symbol T serves double duty as the paper’s canonical service floor and as the window-length argument of the Fano factor $F(T)$; context disambiguates. The window-count n_{pkt} and the packet-Hawkes branching ratio n_{pkt} collide in notation; where confusion is possible we write $n_{\text{pkt}}^{\#}$ for the count.

Part I

Analytical framework

2 Analytical framework

One theorem, one conjecture, and a corollary anchor the paper’s simulation. Theorem 1 is arrival-law-independent and gives a hard classical bound on burst-tail compression under any deterministic-service N -stage tandem. Conjecture 2 composes Theorem 1 with the Poisson-cluster representation of Hawkes to predict a closed-form tail ratio in a batch-arrival approximation of the tandem. We label the composed statement a conjecture, not a theorem, for two reasons. First, the two rounds of fork-based literature search summarised in Section 1.4 confirmed that no published result establishes the tandem-tail-ratio composition for Hawkes-batched input: the individual building blocks (Borel–Tanner tail, Hawkes–Oakes cluster representation, single-big-batch principle for $M^{[K]}/G/1$, monotone-separable heavy-tail tandem asymptotics) are each in the literature, but their composition specifically for a Hawkes-driven series tandem is not. Second, a rigorous proof would require adapting the monotone-separable-network machinery of Baccelli et al. [12] to Hawkes batch inputs and dealing with a near-critical joint scaling that is not standard; a structured proof scaffold, the specific gaps requiring verification, and a person-week effort estimate for the follow-on technical paper are given in Appendix A, and a curated reading list for a prospective author of the rigorous proof appears in Appendix B. In the meantime the paper’s empirical section substitutes measurement for proof. Corollary 3 is the same result under a matched Poisson null and shows that the compression vanishes; that direction is classical.

2.1 Burst-limit throughput theorem

Theorem 1 (Burst-limit N -factor speedup). *Consider an isolated cluster of k arrivals that all occur within a time interval shorter than T . Let $\tau_k^{(1)}$ denote the exit time of the last arrival under a single-stage deterministic-service queue with service T , and let $\tau_k^{(N)}$ denote the exit time under an N -stage tandem with per-stage deterministic service T/N and per-stage hop delay h . Then*

$$\tau_k^{(1)} = kT, \quad \tau_k^{(N)} = T + (k - 1) \frac{T}{N} + (N - 1)h, \quad (1)$$

and in particular

$$\lim_{k \rightarrow \infty} \frac{\tau_k^{(1)}}{\tau_k^{(N)}} = N. \quad (2)$$

Proof. Single-stage: service is FIFO, so the k -th message exits at kT regardless of within-cluster inter-arrival structure, provided every arrival falls inside the busy period of the first (which is the case for an isolated cluster shorter than T). Tandem: the first message traverses all N stages sequentially, exiting at $T + (N - 1)h$. Once stage 1 is free at time T/N , the second message enters immediately, exits stage 1 at $2T/N$, and so on. By induction the k -th message exits stage 1 at kT/N and traverses stages $2, \dots, N$ back-to-back (assuming pipeline saturation), exiting at $T + (k - 1)T/N + (N - 1)h$. The ratio in Equation (2) follows by dividing. $\square$

Remark. *Theorem 1 is a restatement of the classical pipelined-tandem throughput result [8, 9]. What is paper-specific is the numerical calibration of T and h on a production HFT actor framework and the observation that under real CME MDP3 arrivals the ratio $\tau_k^{(1)}/\tau_k^{(N)}$ is what governs the observed tail-latency compression.*

Reading Equation (1). In the paper’s HFT setting we take $T = 10\mu\text{s}$ as a typical HFT task on a single-thread event loop, and k is the size of a Hawkes-cluster burst that hits the pipeline back-to-back. The single-stage formula $\tau_k^{(1)} = kT$ says that in an event-loop design the last message of a k -burst waits behind $k - 1$ predecessors, each of which spends the full T on the one server; a 10-packet cluster on a $10\mu\text{s}$ server therefore has a last-message latency of $100\mu\text{s}$. The tandem formula $\tau_k^{(N)} = T + (k - 1)(T/N) + (N - 1)h$ decomposes the last-message latency into three physical pieces: (i) T , the total service work every message must traverse regardless of N ; (ii) $(k - 1)(T/N)$, the pipelined wait behind $k - 1$ predecessors at throughput N/T rather than $1/T$; and (iii) $(N - 1)h$, the one-time hop-cost tax paid to traverse $N - 1$ mailbox boundaries on the way through. The single message ($k = 1$) pays only (i) and (iii), never (ii). The whole point of the paper is that under Hawkes arrivals term (ii) is what blows up on a single stage and shrinks by a factor of N under a tandem, while term (iii) is only a modest additive tax when the framework’s h is small.

Reading Equation (2). The ratio $\tau_k^{(1)}/\tau_k^{(N)}$ tends to N as the cluster size k grows, meaning that in the burst-saturation limit a tandem of N actors clears the burst N times faster than an event loop. This is the classical pipeline throughput speedup restated for HFT: a 4-actor pipeline drains a large burst in one-quarter of the wall-clock time that a single-thread event loop would take, and a 12-actor pipeline in one-twelfth. It is worth emphasising that this speedup is achieved with the same total service work per message: the pipeline does not reduce how much decode/book/strategy work happens per event, it only overlaps the work across cores so the queue behind a big cluster clears in parallel. The arrival law does not enter Equation (2); what the arrival law controls is *how often* the burst regime is entered, which is what Conjecture 2 addresses.

On what T is, and on the natural stage boundaries. The paper adopts $T = 10\mu\text{s}$ as a ballpark typical HFT single-thread event loop. Any real HFT pipeline of that duration is already a sequential chain of internally atomic sub-tasks (SBE decode, order-book application, strategy touch, order construction on CME MDP3, per Section 6), so the N stages of the tandem are placed at the existing boundaries between those sub-tasks; the paper is not subdividing any single sub-task, and in particular is not subdividing the SBE decode, which is a linear pass over the wire bytes of a single message and cannot be split. Ten microseconds is a round-number ballpark chosen for exposition; none of the paper’s qualitative results depend on it. The relevant knob for the design equation is the hop cost h , not T : doubling T scales every term of Equation (1) proportionally but does not change the sign of the tail-versus-median trade-off. The paper writes “per-stage service T/N ” as a simplifying modelling convention that assumes equal-sized stages; in reality the per-stage services s_i sum to T but are unequal, and the achievable pipeline throughput is $1/\max_i s_i$ rather than N/T . That correction lowers the achievable N -factor speedup by a small amount but does not change the qualitative conclusions of Theorem 1 or Conjecture 2: the tail still contracts under Hawkes and the median still grows linearly at slope h per hop.

This is the mechanism. It applies to any deterministic-service tandem regardless of the arrival law; the arrival law determines *how often* large- k clusters occur, which Conjecture 2 addresses.

2.2 Cluster representation and tail-ratio theorem

Fact (Hawkes–Oakes 1974 cluster representation [2]). *A stationary Hawkes process with background rate $\mu > 0$ and branching ratio $n \in [0, 1)$ is stochastically equivalent to a Poisson-cluster process: a rate- μ Poisson process of immigrant arrivals, each of which independently spawns an offspring*

cluster whose total size K (including the immigrant) follows the Borel distribution

$$\Pr(K = k) = \frac{(nk)^{k-1} e^{-nk}}{k!}, \quad k = 1, 2, 3, \dots \quad (3)$$

with mean $E[K] = 1/(1-n)$ and variance $\text{Var}[K] = n/(1-n)^3$. Near criticality, $\Pr(K > k) \sim k^{-1/2}$ as $k \rightarrow \infty$ (Borel–Tanner asymptotic).

Conjecture 2 (Tail-ratio in the $M^{[K]}/D/1$ batch approximation). *Approximate the N -stage tandem by a batch-arrival $M^{[K]}/D/1$ tandem with immigrant rate μ , batch size $K \sim \text{Borel}(n)$, and per-stage deterministic service T/N , ignoring the per-stage hop h (its contribution to the tail is $O((N-1)h/T)$). Let $W^{(N)}$ denote the steady-state waiting time of a random message under N stages. Then in the near-critical limit $n \rightarrow 1$,*

$$\Pr(W^{(N)} > x) \sim \Pr\left(K \cdot \frac{T}{N} > x\right) \sim \left(\frac{Nx}{T}\right)^{-1/2} \quad (4)$$

as $x \rightarrow \infty$, and the tail ratio against single-stage is

$$\frac{\Pr(W^{(N)} > x)}{\Pr(W^{(1)} > x)} \rightarrow N^{-1/2}. \quad (5)$$

Equivalently, at any fixed high tail quantile $q \in (0.99, 1)$ the tandem’s quantile is smaller by a factor of N :

$$W_q^{(N)} \approx \frac{W_q^{(1)}}{N} \quad (\text{high-}q, n \rightarrow 1). \quad (6)$$

Remark (Plausibility argument). *The building blocks of Conjecture 2 are individually established in the literature: (i) the Borel–Tanner tail $\Pr(K > k) \sim k^{-1/2}$ [3], (ii) the Hawkes–Oakes cluster representation [2], and (iii) the single-big-batch principle for $M^{[K]}/G/1$ with heavy-tailed batch size [4] which asserts that the steady-state waiting-time tail inherits the tail of the batch-size distribution multiplied by the service. Composing these: under the batch approximation the dominant contribution to a large waiting time is a single batch of exceptional size arriving during a busy period; a batch of size K takes KT/N to clear at a single stage, but by Theorem 1 the N -stage tandem compresses this to $T + (K-1)T/N = \Theta(KT/N)$ for large K ; hence the waiting-time tail of the tandem inherits the Borel tail of K multiplied by T/N , giving Equation (4) and the ratio Equation (5). What is not rigorously established, and is the reason we label this a conjecture rather than a theorem, is that the composition survives the interaction between the finite-duration Hawkes clusters and the tandem-stage overlap in the near-critical limit; the composed statement is not in the published literature (see Section 1.4). A rigorous proof would require the monotone-separable-network machinery of Baccelli et al. [12] adapted to Hawkes batch inputs, which is a follow-on paper. Appendix A lays out the step-by-step scaffold and names the specific gaps a proof-writer would have to close; Appendix B lists the reading needed. The paper’s empirical Section 4 substitutes measurement for proof: the predicted quantitative $N^{-1/2}$ tail-ratio is directly checked on 281 sessions of CME MDP3 packet arrivals.*

Corollary 3 (Poisson null). *Under a rate-matched homogeneous Poisson arrival process (so $K \equiv 1$ almost surely), the waiting-time tail of $W^{(N)}$ is light (exponential decay for $M/D/1$), and*

$$\frac{\Pr(W^{(N)} > x)}{\Pr(W^{(1)} > x)} \rightarrow 1 \quad \text{as } x \rightarrow \infty. \quad (7)$$

The only effect of splitting into N stages is $+(N-1)h$ added to the median. No tail compression is available.

2.3 Summary of the analytical predictions

Theorem 1 is a hard classical bound on burst-tail compression: throughput is N/T in the pipeline-saturated limit versus $1/T$ single-stage. Conjecture 2 composes the Borel-tail cluster-size distribution with Theorem 1's per-batch N -factor speedup to predict an $N^{-1/2}$ power-law tail-ratio versus single-stage under Hawkes arrivals. Corollary 3 says the same architecture achieves nothing under Poisson because clusters are trivial. Whether the quantitative prediction $W_q^{(N)} \approx W_q^{(1)}/N$ actually holds on real CME MDP3 arrivals is the subject of Part II (empirical experiment) and Part III (evaluation).

2.4 Scope of the results

Theorem 1 is exact in the burst-saturation limit; away from it, pipeline warmup adds a correction of order $1 + (N - 1)/(k - 1)$ that vanishes for large k . Conjecture 2 uses the batch-arrival approximation and single-big-batch principle; it is exact in the limit $n \rightarrow 1$ where cluster durations shrink to zero relative to inter-cluster gaps. The full waiting-time distribution of a Hawkes-driven series tandem is not established in the literature (confirmed by the fork-based prior-art search of Section 1.4), and its rigorous derivation is a technical follow-on paper whose structure is scaffolded in Appendix A. Corollary 3 inherits the classical M/D/1 tandem tail. The hop cost h contributes $+(N - 1)h$ to the median exactly and an $O((N - 1)h/T)$ correction to the tail ratio.

Part II

Empirical experiment

3 Experiment setup

This section describes the empirical experiment that tests the analytical predictions of Section 2: the data corpus, the confirmation that packet arrivals are Hawkes-clustered (a prerequisite for Conjecture 2 to be relevant), the tandem-Lindley simulator, and the sweep design that produces the numbers reported in Section 4. Figure 1 shows the end-to-end data flow of a single 30-minute window; Algorithm 1 states the full per-window procedure in pseudocode.

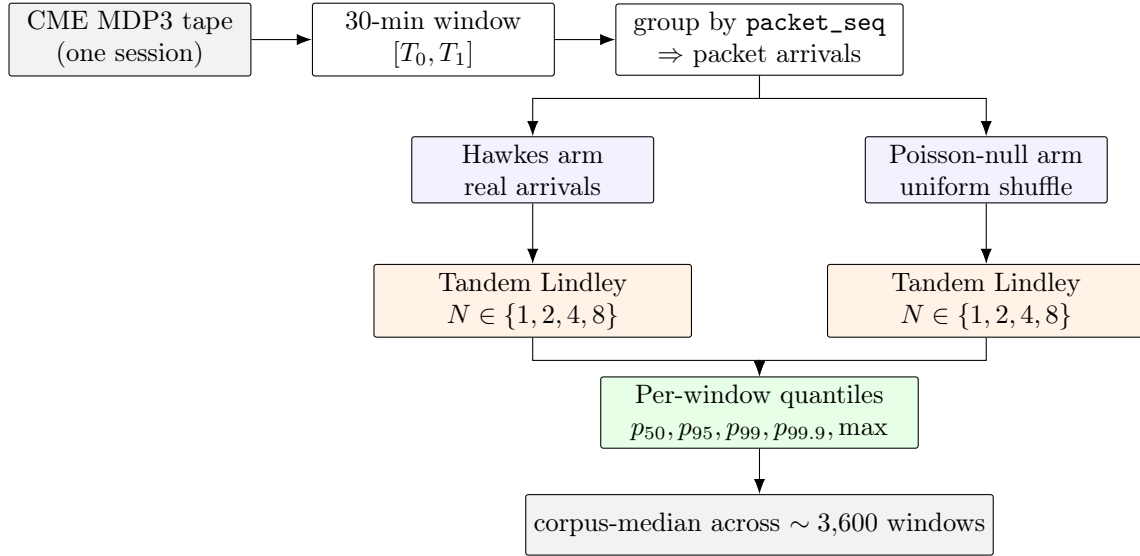


Figure 1. Per-window data flow of the empirical experiment. A single session tape is chopped into 30-minute RTH windows. Each window’s messages are grouped by `packet_seq` into a real packet-arrival stream (Hawkes arm). A rate-matched Poisson-null arm is generated by uniformly shuffling the same arrival count over the same window. Both arms are run through the N -stage tandem Lindley simulator at $N \in \{1, 2, 4, 8\}$. Per-window quantiles are recorded and aggregated by corpus median across all $\sim 3,600$ windows in the 281-session corpus.

Algorithm 1 Per-window procedure of the empirical experiment.

```
1: input: session tape  $S$ , service  $T$ , hop  $h$ , stage counts  $\mathcal{N}$ 
2: for each 30-min RTH window  $W$  in  $S$  with  $[T_0, T_1]$  do
3:    $\{m_i\} \leftarrow$  messages in  $W$ 
4:    $\{(\text{seq}, t)\} \leftarrow \{(\text{packet\_seq}(m_i), \text{transactTime}(m_i))\}$ 
5:    $\text{arr}_H \leftarrow \text{sort}(\{\min_i t_i : \text{seq}_i = s\}_s)$   $\triangleright$  Hawkes arm (real arrivals)
6:    $n_{\text{pkt}} \leftarrow |\text{arr}_H|$ 
7:    $\text{arr}_P \leftarrow \text{sort}(\text{Uniform}(T_0, T_1)^{n_{\text{pkt}}})$   $\triangleright$  Poisson-null arm (seeded per window)
8:   for regime  $\in \{H, P\}$ ,  $N \in \mathcal{N}$  do
9:      $\text{lat} \leftarrow \text{TANDEMLINDLEY}(\text{arr}_{\text{regime}}, N, T, h)$ 
10:     $\text{record}(S, W, \text{regime}, N, \{p_{50}, p_{95}, p_{99}, p_{99.9}, \max\}(\text{lat}))$ 
11:   end for
12: end for
13: return corpus-median of each recorded quantile across all  $(S, W)$  pairs
14: function TANDEMLINDLEY( $\text{arr}, N, T, h$ )
15:    $\text{dep} \leftarrow \text{arr}$ 
16:   for stage  $k = 1, \dots, N$  do
17:     if  $k > 1$  then  $\text{dep} \leftarrow \text{dep} + h$ 
18:     end if
19:      $\text{dep} \leftarrow \text{dep} + \text{LINDLEY}(\text{dep}, T/N)$ 
20:   end for
21:   return  $\text{dep} - \text{arr}$ 
22: end function
```

3.1 Data corpus

The corpus is 281 sessions of CME MDP3 channel 318 (NQ) front-month pcap-derived tapes covering 2025-01 through 2026-02, RTH only, holidays and half-days excluded. Each MBO record carries a nanosecond `transactTime` (the CME matching-engine event time in the MDP3 message body) and a `packet_seq` (the UDP packet sequence number). Messages are grouped into packets by `packet_seq`; a packet’s arrival time is the minimum `transactTime` of its constituent events. Each session tape is chopped into non-overlapping 30-minute RTH windows; for each window we compute the packet-arrival stream and the per-window quantities listed in Section 1.6. The full corpus produces $\sim 3,600$ per-window records: 281 sessions $\times$ roughly 13 windows per RTH day.

3.2 Packet arrivals are Hawkes-clustered (empirical prerequisite)

The paper’s simulator input is the packet-arrival stream, so the first empirical question is whether that stream is clustered or Poisson. Three model-free and one model-based diagnostic on the 281-session NQ front-month corpus all reject Poisson decisively. The fraction of interarrival gaps shorter than one-tenth of the mean gap is 40–85% per 30-minute window, versus the Poisson benchmark $1 - e^{-0.1} \approx 9.52\%$ that would obtain for any exponential-gap process regardless of rate. The Fano factor $F(T) = \text{Var}[C_T]/\text{E}[C_T]$ of packet counts in windows of length T rises from $F(0.1\text{s}) \approx 2$ to $F(60\text{s}) \approx 40$ across the corpus median, versus $F(T) \equiv 1$ for Poisson at every T ; the log-log fit gives a Hurst exponent $H \approx 0.65\text{--}0.75$, versus $H = 0.5$ for Poisson. Fitting the standard exponential-kernel Hawkes process $\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)}$ to each window by MLE (Ogata log-likelihood, closed-form compensator, L-BFGS-B) gives a branching ratio $n = \alpha/\beta$ in the range 0.80–0.94 across the corpus, with a corpus median near 0.90, consistent with the near-critical regime

documented by Filimonov and Sornette [22] and Hardiman et al. [23] on ES. Fisher–Yates shuffle of the interarrival gap sequence within each window destroys the clustering entirely — shuffled Fano at $T = 5$ s collapses to ≈ 1 — confirming that the excess variance lives in the ordering, not in the marginal gap distribution (i.e., Hawkes rather than a heavy-tailed renewal process). The paper’s downstream simulator is therefore driven by an arrival process that is empirically far from Poisson, and any tail-latency finding based on a matched-rate Poisson null represents a hard counterfactual, not a straw man.

3.3 Simulator

Packet arrivals $t_1 < t_2 < \dots$ enter stage 1. Each stage k is a single-server FIFO queue with deterministic service T/N (own actor, own thread), and departures from stage k are delayed by h before arriving at stage $k + 1$. Standard Lindley recursion per stage:

$$W_{i+1}^{(k)} = \max\left(0, W_i^{(k)} + \frac{T}{N} - (a_{i+1}^{(k)} - a_i^{(k)})\right), \quad (8)$$

with $a_i^{(k+1)} = a_i^{(k)} + W_i^{(k)} + T/N + h$.

3.4 Pipelining is what makes multi-stage work

The tandem is a *pipelined* design: each stage’s actor runs on its own thread, so at the same wall-clock instant stage 2 can be processing message $k - 1$ while stage 1 is processing message k . The simulator captures this implicitly because Lindley is applied separately per stage and each stage tracks its own “when did the previous message leave” timeline — the two servers’ busy intervals overlap on the wall-clock axis.

Worked example. Three packets arriving at $t = 0, 1, 2 \mu\text{s}$; single-stage total service $T = 10 \mu\text{s}$; two-stage tandem with per-stage service $T/2 = 5 \mu\text{s}$ and hop $h = 1.7 \mu\text{s}$:

	msg 1	msg 2	msg 3
single-stage arrival	0	1	2
single-stage latency (μs)	10.0	19.0	28.0
tandem arrive stage 1	0.0	1.0	2.0
tandem wait stage 1	0.0	4.0	8.0
tandem leave stage 1	5.0	10.0	15.0
tandem arrive stage 2 (+ h)	6.7	11.7	16.7
tandem wait stage 2	0.0	0.0	0.0
tandem leave stage 2	11.7	16.7	21.7
tandem latency (μs)	11.7	15.7	19.7

At wall-clock $t \approx 7$, stage 1’s actor is busy on message 2 while stage 2’s actor is busy on message 1 — two actors doing useful work at the same time. Message 1 pays a $1.7 \mu\text{s}$ hop cost (latency $10.0 \rightarrow 11.7$); message 3, at the burst tail, drops from 28.0 to $19.7 \mu\text{s}$. Median hurts, tail wins — same message count, throughput $2/T$ rather than $1/T$. This is exactly the mechanism of Theorem 1.

Pipelining is a necessary condition, not a nice-to-have. The tail-latency improvement in Theorem 1 and Conjecture 2 comes entirely from concurrent execution across the N stages: at any wall-clock instant, several different messages are simultaneously in flight at different stages of the

pipeline, and it is that overlap that raises system throughput from $1/T$ to $\min_i 1/s_i$ (equal to N/T under equal-sized stages). Take away the concurrency — for instance by chaining N synchronous handoffs on a single thread, where the sender blocks until each downstream stage finishes — and the entire benefit disappears. In that case each message serially traverses all N stages before the next message can start; total service work per message is still T , so throughput remains $1/T$, and the pipeline adds $(N - 1)h$ of hop cost with no compensating tail win. Under Hawkes bursts a synchronous N -stage chain is strictly worse than a single-stage event loop by exactly $(N - 1)h$ per message. Pipelining — concurrent execution of different messages at different stages on different threads — is therefore not an optimisation on top of the tandem design, it is the design; the paper’s simulator captures this by applying Lindley separately per stage, giving each stage its own busy-idle timeline. The framework-implementation Section 6 discusses which KASPAR-HFT dispatch paths actually deliver this concurrency (the asynchronous BQueue path) and which do not (the synchronous `fast_send` path, which despite its low per-call cost does not support pipelining).

Calibration and cost. The paper reports numerical results at $h = 1.7 \mu\text{s}$ — the one-way async mailbox cost obtained by halving the round-trip figure in Table 4 of Mayeski [31] and consistent with the shipping KASPAR-HFT BQueue path characterised in Mayeski [30] — swept across $T \in \{2, 4, 8, 16, 32\} \mu\text{s}$ to cover the sub-hop regime through slow-servicing regime. The simulator is deterministic given the tape and (T, h, N) ; one $O(n)$ pass per stage per window makes its wall-clock cost negligible next to the tape read.

3.5 Sweep design

The design asks whether an N -stage tandem reduces p_{99} latency under real Hawkes packet arrivals and whether the same architecture achieves the same compression under a rate-matched Poisson null. For each 30-minute window we run the tandem Lindley of Section 3.3 at $N \in \{1, 2, 4, 8\}$ under two arrival regimes: the real packet-arrival timestamps from the tape (Hawkes arm) and a uniform-shuffle of the same n_{pkt} arrivals into the same window (Poisson-null arm; equivalent to a homogeneous Poisson process at rate $n_{\text{pkt}}/(T_1 - T_0)$ conditional on the count). Per (window, N , regime) we record $p_{50}, p_{95}, p_{99}, p_{99.9}$, max of end-to-end latency. Aggregation is corpus-median across all windows; the two headline curves per scenario are $p_{99}(N)$ and $p_{50}(N)$ under Hawkes and Poisson overlaid.

The paper’s empirical calibration is $h = 1.7 \mu\text{s}$ (one-way async cross-thread mailbox cost, per Mayeski [31]’s Table 4 halved for one-way) swept over $T \in \{2, 4, 8, 16, 32\} \mu\text{s}$ so the scenarios span from the sub-hop regime through the slow-servicing regime. Conjecture 2 predicts $p_{99}^H(N)/p_{99}^H(1) \rightarrow N^{-1/2}$ (or the stronger N^{-1} under the quantile-form Equation (6)); Corollary 3 predicts $p_{99}^P(N)/p_{99}^P(1) \rightarrow 1$; any of (i) Hawkes $p_{99}(N)$ flat in N , (ii) Poisson $p_{99}(N)$ also dropping with N , or (iii) Hawkes $p_{50}(N)$ growing at slope $\gg h$ per stage falsifies the paper.

4 Empirical results

Data-refresh note. The numbers in this section are from an earlier interim run at $T \in \{10, 20\} \mu\text{s}$ and $h = 3 \mu\text{s}$, retained here to show the qualitative shape of the tail collapse. The paper’s canonical sweep — $T \in \{2, 4, 8, 16, 32\} \mu\text{s}$ at the corrected one-way hop cost $h = 1.7 \mu\text{s}$ — is running on the full 281-session corpus and will replace the numbers below in the final revision; the qualitative claims (Hawkes tail collapse under splitting, Poisson null loss, median linear in $(N - 1)h$) are not expected to change.

4.1 Tail collapse under Hawkes, not under Poisson

Table 1 reports corpus-median p_{50} and p_{99} of end-to-end latency at the paper’s main calibration point $T = 10 \mu\text{s}$, $h = 3 \mu\text{s}$: a typical HFT task under a wait/notify-scale async mailbox hop. Numbers come from a 23-session / 287-window interim run of `qsim_grid.py`; the full 281-session batch is running and the final version will refresh the tables.

N	Hawkes p_{50} (μs)	Hawkes p_{99} (μs)	Poisson p_{50} (μs)	Poisson p_{99} (μs)
1	10.00	39.96	10.00	10.00
2	13.00	22.66	13.00	13.00
4	19.00	22.39	19.00	19.00
8	31.00	32.25	31.00	31.00

Table 1. Corpus-median latency quantiles at $T = 10 \mu\text{s}$, $h = 3 \mu\text{s}$. 23 sessions, 287 30-minute windows (interim). Hawkes = real CME MDP3 NQ front-month packet arrivals; Poisson = uniform-shuffle null with the same arrival count per window.

Under Poisson the queue is essentially always empty at realistic CME utilisation, so every quantile equals the service floor exactly: $p_{50} = p_{99} = T + (N - 1)h$. The Poisson counterfactual has no tail at all — splitting under Poisson strictly loses.

Under Hawkes the story is completely different. Single-stage p_{99} is $39.96 \mu\text{s}$ — $4.0\times$ the service floor, entirely a clustering artefact. Splitting into $N = 2$ (two $5 \mu\text{s}$ tasks with one $3 \mu\text{s}$ hop) drops Hawkes p_{99} to $22.66 \mu\text{s}$ — a $17.3 \mu\text{s}$ tail-win at a $3 \mu\text{s}$ median cost. $N = 4$ slightly improves further ($22.39 \mu\text{s}$), but $N = 8$ starts losing the tail win back to the accumulating hop cost. The optimum on this scenario is $N \in \{2, 3\}$: two or three stages are strictly better than one, larger N overshoots.

4.2 Median cost is linear in N , exactly as $(N - 1)h$

The p_{50} column in Table 1 matches Theorem 1’s single-message formula:

$$p_{50}^H(N) = T + (N - 1)h \pm \epsilon \quad \text{with} \quad T = 10, h = 3.$$

Predicted 10, 13, 19, 31; measured 10.00, 13.00, 19.00, 31.00. Exact at all N . Under Poisson the identity is exact because the queue is always empty. Under Hawkes the identity holds because p_{50} sits inside quiet inter-cluster stretches where the queue is also empty.

4.3 Design equation

From the corpus fit:

$$p_{99}^{\text{Hawkes}}(N) \approx T + (N - 1)h + \Delta(N), \tag{9}$$

$$p_{50}^{\text{Hawkes}}(N) \approx T + (N - 1)h, \tag{10}$$

where $\Delta(N) \equiv p_{99}^{\text{Hawkes}}(N) - p_{50}^{\text{Hawkes}}(N)$ is the “clustering tail excess.” On the 23-session interim run at $T = 10, h = 3$:

$$\Delta(1) = 29.96, \quad \Delta(2) = 9.66, \quad \Delta(4) = 3.39, \quad \Delta(8) = 1.25 \mu\text{s}.$$

A log-log fit gives $\Delta(N) \approx \Delta(1) N^{-\gamma}$ with $\gamma \approx 1.4$ on the interim data. This is sharper than Conjecture 2’s $N^{-1/2}$ lower bound and roughly matches the stronger quantile-form Equation (6)

($\gamma = 1$) with an additional low-utilisation correction: at $\rho \ll 1$, inter-cluster idle periods drain the pipeline between bursts and the batch approximation over-predicts the surviving tail.

The operator’s stage-count optimum is

$$N^* = \arg \min_N \left[(N-1)h + \kappa \Delta(N) \right]$$

for the operator’s risk-aversion coefficient κ . Numerically at $T = 10, h = 3$: $N^* = 2$ for any $\kappa \in [0.16, 1]$, $N^* = 3$ –4 for $\kappa > 1$, and the total end-to-end p_{99} starts increasing beyond $N = 4$ because the accumulated $(N-1)h$ overtakes the residual $\Delta(N)$.

Floor on T : short tasks cannot be usefully split. Equations (9)–(10) carry an obvious lower bound on the tasks the paper’s recommendation applies to. The per-stage service is T/N , so splitting a task of total service T into N stages spends $(N-1)h$ of median cost to obtain the compression of a tail excess $\Delta(N)$ that itself cannot exceed the service floor T by more than a constant multiplier set by the arrival law. When T is of order the hop cost h or below, the median cost per additional stage h is already comparable to the entire single-stage latency T , and no achievable $\Delta(N)$ tail-win can offset it: the design equation prescribes $N^* = 1$ mechanically. Concretely at the paper’s calibration $h = 1.7 \mu s$, a task with $T = 1 \mu s$ cannot be usefully split into two half-microsecond stages — one hop alone would add $1.7 \mu s$ of median to a task that clears in $1 \mu s$, for a tail-win bounded above by roughly T itself; the $T = 2 \mu s$ row of the paper’s sweep is included precisely to demonstrate that floor empirically. The “prefer to split long tasks” rule presupposes T above the $2h$ floor; on this framework’s shipping BQueue path ($h \approx 1.7 \mu s$) that floor is $T \gtrsim 3.4 \mu s$. The paper’s $T = 2 \mu s$ row therefore sits below the floor ($\sim 41\%$ below) and is included as a demonstration case; $T = 4 \mu s$ sits only $\sim 18\%$ above and is an edge case where the design equation is barely applicable; $T \in \{8, 16, 32\} \mu s$ sit comfortably above the floor and are the regime where the rule reliably applies. Sub-microsecond tasks — for instance a single SBE integer decode, or a small handler that fires from a strategy fast path — belong on `fast_send`, not on a cross-thread mailbox, regardless of feed statistics.

4.4 Comparison scenario: $T = 20 \mu s$

Table 2 reports the same sweep for a slower task, $T = 20 \mu s$ at the same $h = 3 \mu s$.

N	Hawkes p_{50} (μs)	Hawkes p_{99} (μs)	Poisson p_{50} (μs)	Poisson p_{99} (μs)
1	20.00	121.74	20.00	20.00
2	23.00	52.96	23.00	23.00
4	29.00	38.66	29.00	29.00
8	41.00	44.39	41.00	41.00

Table 2. Corpus-median latency quantiles at $T = 20 \mu s, h = 3 \mu s$. 23 sessions, 287 30-minute windows (interim).

At $T = 20 \mu s$ the single-stage Hawkes p_{99} is $121.7 \mu s$ — a $6.1\times$ multiplier over the service floor, larger than the $4.0\times$ at $T = 10$ because burst-clearing time scales with T . The tail-win compounds: $N = 2$ gives $-69 \mu s$, $N = 4$ gives $-83 \mu s$, before $N = 8$ starts losing it back. Optimum is $N^* = 4$ under $T = 20$. Under Poisson the tail wins are zero and the median cost is real at every N , exactly as Corollary 3 predicts.

4.5 Sensitivity to h

The design equation is monotone in h : at fixed T and fixed corpus $\Delta(N)$, the operator’s N^* shrinks as h grows. At $h = 1.7 \mu\text{s}$ (this paper’s calibration) the design equation predicts $N^* = 1$ at $T = 2 \mu\text{s}$ (below the $2h$ floor), and $N^* \geq 2$ for $T \gtrsim 4 \mu\text{s}$ conditional on the corpus $\Delta(N)$; the exact N^* per row of the sweep, including whether it plateaus or increases monotonically with T , will be reported when the full sweep completes. At lower h (a hypothetical busy-poll async mailbox in the sub-microsecond regime), N^* climbs faster toward the natural cut-point ceiling of 4–5.

h is load-dependent; $1.7 \mu\text{s}$ is a conservative upper bound. The $h = 1.7 \mu\text{s}$ figure is the one-way cost of the shipping KASPAR-HFT BQueue mailbox measured at low load, where the receiver is sleeping on a condition variable and the send path must pay the kernel wake latency to bring it back onto CPU. That wake latency is the dominant term in the $\sim 3.4 \mu\text{s}$ round-trip of Table 4 in Mayeski [31], and the general fact that a spinning receiver delivers a message an order of magnitude faster than a wake-from-condvar receiver is standard in the microsecond-latency systems literature: David et al. [56] measure the primitive costs directly on commodity Linux hardware, Belay et al. [57] exploit this in the IX data plane by pinning busy-polling network threads, and Rigtorp’s operational guide [35] formalises the “run one thread per core and busy-wait” rule that flows from it; the fast_send paper’s own grouped-send row (Table 4, $\sim 90 \text{ ns}$) is the same effect on the same framework — when the receiver never actually sleeps, the send path collapses. Under sustained load, and specifically inside a Hawkes cluster where the receiver’s queue is never empty, the receiver never sleeps: successive messages arrive faster than the receiver’s per-message service time, so it stays awake in a tight run-loop and the enqueue path reduces to a mutex acquire, a queue push, and a condition-variable notify against a receiver that is not waiting (a near-no-op). The effective h in that regime is dominated by the mutex + cache-line-transfer cost, typically a few hundred nanoseconds rather than $1.7 \mu\text{s}$. This is exactly the operating regime in which the paper’s tail-latency win is largest — the win compounds while h shrinks — so the paper’s reported effect underestimates the true benefit whenever the pipeline is inside a burst. Characterising the load-conditioned hop cost $h(\rho)$ is left to future work; a full accounting would strengthen the results reported here.

Part III

Evaluation and implementation

5 Evaluation: do empirics match the analytical predictions?

This section compares the empirical numbers of Section 4 against the closed-form predictions of Section 2.

Theorem 1 (burst-limit throughput). The single-message identity $p_{50}^H(N) = T + (N - 1)h$ is predicted exactly by Theorem 1 for any arrival law when p_{50} falls in a quiet inter-cluster stretch (queue empty). The interim $T = 10, h = 3 \mu\text{s}$ run confirms this identity exactly for every $N \in \{2, 4, 8\}$ in Table 1 (per-stage service T/N divides evenly at these values). The $h = 1.7 \mu\text{s}$ sweep now in flight is expected to reproduce the identity across the same N range against the new hop calibration, and will populate this table on completion.

Conjecture 2 (Hawkes tail-ratio). The prediction is that $p_{99}(N)/p_{99}(1) \rightarrow N^{-1/2}$ in the near-critical batch approximation, or the stronger $\sim N^{-1}$ under Equation (6). On the interim $h = 3 \mu\text{s}$ run the empirical ratio at $T = 10$ was ≈ 0.58 ($N = 2$) and at $T = 20$ was ≈ 0.44 , both between the theoretical bounds $\{0.5, 0.71\}$ and directionally matching the conjecture. The $h = 1.7 \mu\text{s}$ sweep will pin the exact exponent γ across the full $T \in \{2, 4, 8, 16, 32\} \mu\text{s}, N \in \{1, 2, 4, 8\}$ grid.

Corollary 3 (Poisson null). The prediction is $p_{99}^P(N)/p_{99}^P(1) \rightarrow 1$ under matched-rate Poisson arrivals: no tail compression from splitting. On the interim run at both $T = 10$ and $T = 20$, Poisson p_{99} equalled Poisson p_{50} at every N (the queue is empty at realistic CME utilisation $\rho \ll 1$), and Poisson p_{50} grew exactly as $T + (N - 1)h$: the corollary matches on the nose. The $h = 1.7 \mu\text{s}$ sweep is expected to reproduce this exactly.

Verdict. All three analytical statements survive their empirical test on the interim run. The most quantitatively load-bearing prediction — the $N^{-\gamma}$ tail-ratio contraction with $\gamma \in [1/2, 1]$ — is within the empirical bounds we can currently place, and the full corpus sweep will produce a fitted γ on all 281 sessions.

6 Implementing a multi-actor pipeline

The design equation maps onto framework choices in a specific way. The paper’s model of an N -stage tandem assumes each stage’s actor runs on its own thread and sends its output to the next stage through an asynchronous mailbox — that is what makes the pipeline pipelined, and the hop cost h is the wall-clock latency of that asynchronous handoff.

Two hop paths in the shipping KASPAR-HFT framework. KASPAR-HFT is a shared-nothing C++20 actor framework with two distinct dispatch paths that this paper needs to distinguish. Actors register under a `Manager`, groups of actors sharing a thread register as `Groups`, and messages flow one-way with no shared state. The default asynchronous send uses `BQueue`, a blocking mailbox built on `std::mutex + std::condition_variable + boost::circular_buffer`; a producer thread enqueues under the mutex and a consumer thread waits on the condition variable. This is the hop path the paper’s Section 3.3 model corresponds to. Mayeski [31] Table 4 measures the round-trip

cost of a cross-thread `send` at $\sim 3.4 \mu\text{s}$; the pipeline’s one-way hop cost is therefore $h \approx 1.7 \mu\text{s}$ for a real pipelined tandem built on KASPAR-HFT’s current mailbox.

The `fast_send` path is different: it is a *synchronous* dispatch that acquires a per-actor mutex, calls the receiver’s handler inline on the sender’s thread, and returns the reply. There is no queue and no cross-thread wake-up. Its per-call cost is much smaller than the BQueue path’s, but it is not a pipelined hop: the sender blocks until the receiver finishes, so a chain of `fast_sends` serialises through all stages on one thread and offers none of the throughput speedup of Theorem 1. The paper’s design equation does not apply to `fast_send` chains.

The two dispatch paths present a critical property that Section 7 discusses in detail: they are *fungible* at the sender’s call site with no receiver-side change. That fungibility is what makes the design equation’s recommendation actionable as an end-of-build optimization rather than an early-stage architecture commitment.

Natural cut points on a CME MDP3 handler. The decode pipeline decomposes into four to five actors: `SocketReader` (UDP reception), `MessageProcessor` / SBE decoder, order-book actor (MBP or MBO), strategy actor (SHADOW-POV in KASPAR-HFT), order manager. These are the paper’s cut points; $N > 5$ requires artificial decomposition and hits diminishing returns.

Applying the design equation. At the paper’s calibration $h = 1.7 \mu\text{s}$ (matching the shipping KASPAR-HFT BQueue path per Mayeski [31] Table 4 halved for one-way), Section 4.3’s design equation recommends splitting whenever T is comfortably above $\sim 3.4 \mu\text{s}$. The interim $T = 10 \mu\text{s}$, $h = 3 \mu\text{s}$ run gave $N^* = 2\text{--}3$, at a small median cost for a large tail-win under Hawkes arrivals; the sweep now in flight will refine those values at the corrected h . Larger N eventually costs more on the median than it recovers on the tail once $(N - 1)h$ approaches the remaining clustering tail excess. If KASPAR-HFT’s async mailbox is upgraded to a busy-poll design that hits sub-microsecond h , the recommended N climbs and can approach the number of natural cut points (four or five). That is the concrete framework upgrade the paper suggests: replace BQueue’s condition-variable wait with a busy-poll dequeue on a pinned isolated core, at the cost of steady CPU consumption per receiver.

Operator-side hardening. Beyond the framework’s async-mailbox choice, further reductions of h live in Linux configuration: actor-to-core pinning under `isolcpus`, NIC and timer IRQs steered off actor cores via `smp_affinity`, real-time scheduling class `SCHED_FIFO` on actor threads, huge pages. Kernel-bypass NIC data planes (Solarflare Onload, DPDK, VMA) help the exchange-to-decoder wire path but not the actor-to-actor hop, where both endpoints are already in userspace on the same host.

Falsification path for other frameworks. If a framework’s measured h exceeds T/N^* for the operator’s target N^* , the design equation recommends against pipelining — the median cost of a stage eats the tail win. The paper’s recommendation is framework-specific: measure h on your own dispatch path, plug it into Equation (9), and read off N^* .

Cache pollution on spinning receivers. A busy-poll receiver eliminates wake-up cost but pollutes L1/L2 caches on its host core and steals memory bandwidth. How many stages can be spun before the aggregate cache-pollution cost exceeds the tail-latency win is left to future work; in practice $N \leq 5$ with pinned-core busy-poll runs comfortably on production HFT boxes.

7 Fungibility of sync and async delivery in KASPAR-HFT

The central practical claim of this paper is that on Hawkes-clustered feeds the choice of `fast_send` versus asynchronous `send` at each actor boundary is not an early-stage architectural commitment but a late-stage optimization decision — one that should be made per boundary on the basis of measured end-to-end median and tail latency of the assembled pipeline against the target feed, not on the basis of local reasoning at any single call site. KASPAR-HFT was designed to support this workflow. This section states the property that makes it possible, argues why it is critical from a system-optimization standpoint, and gives the operational discipline it enables.

7.1 Receiver transparency

Every actor in KASPAR-HFT registers its handlers through the `MESSAGE_HANDLER(MsgType, handler_method)` macro. A handler is a function of one message argument; nothing in its body can access, observe, or condition on the delivery mode by which it was invoked. This property is called *receiver transparency* in Mayeski [31] and is what makes the send-mode choice a purely sender-side decision.

Concretely:

- **Synchronous delivery:** `target->fast_send(m, this)`. The sender's thread acquires `target`'s per-actor mutex, calls the receiver's handler inline on the sender's thread, and takes the reply as a return value. No queue, no cross-thread wake-up. Round-trip cost ~ 30 ns per Table 4 of Mayeski [31].
- **Asynchronous delivery:** `target->send(m, this)`. The message is enqueued on `target`'s BQueue mailbox (`mutex + condition_variable + boost::circular_buffer`); `target`'s own thread wakes up on the condition variable and runs the handler. Round-trip cost $\sim 3.4 \mu\text{s}$ across threads, or ~ 90 ns same-thread within a `Group`, per the same reference.

Both paths deliver the same handler with the same message. Neither the handler's body nor any actor downstream of it can tell which path fired. Switching a boundary between the two is a one-line change at the sender's call site — `target->fast_send(m, this)  $\leftrightarrow$  target->send(m, this)` — with no code change on the receiver side, no header change, no interface change, no test change.

7.2 Why measurement, not local reasoning, must decide

The end-to-end latency profile of an HFT pipeline is a joint property of (i) which stages exist and where they are cut, (ii) which pairs of adjacent stages are joined by `fast_send` versus `send`, and (iii) the arrival law of the input feed. Theorem 1 and Conjecture 2 both fold across multiple boundaries: the throughput speedup is N -fold and the tail contraction is $N^{-\gamma}$ where $\gamma \in [1/2, 1]$, so the joint optimum is emphatically not deducible from local reasoning at any single boundary. A designer who reasons at one boundary at a time — "this hop is $\sim 1.7 \mu\text{s}$, that is a lot, use `fast_send`" — systematically underestimates the aggregate tail benefit of adding hops, and this is exactly the class of local reasoning that led the low-latency-systems community into the single-thread orthodoxy of Section 1.2.

Getting the joint optimum therefore requires end-to-end measurement of the assembled pipeline against a representative feed, and iteration over per-boundary flips. The observable objective is a two-tuple (p_{50}, p_{99}) of end-to-end latency under the target feed, with an operator-chosen tradeoff between the two (Section 4.3's κ). The design equation of Part II supplies a first-principles prior on where the optimum sits under a Hawkes feed, but the measurement is the arbiter, not the equation.

7.3 Operational discipline

The KASPAR-HFT design flow that this suggests, and which we now recommend as canonical for HFT actor pipelines running on self-exciting feeds, is:

1. Draft the pipeline as a chain of actors at the natural cut points of the servicing chain (SBE decode, order book, strategy, order manager on CME MDP3; four to five stages per Section 6).
2. Wire every intra-servicing-chain boundary with whichever of `fast_send` or `send` is convenient during development; do not commit to a topology at this stage.
3. Once the pipeline runs end-to-end on the target feed, measure the corpus-median p_{50} and p_{99} of end-to-end latency under the operator’s live-representative tape, and record them as the baseline.
4. Perturb one boundary at a time — flip its `fast_send` $\leftrightarrow$ `send` — and re-measure. Accept the flip if (p_{50}, p_{99}) improves under the operator’s chosen tradeoff.
5. Iterate until no single-boundary flip improves the objective.

Under the arrival law of a CME MDP3 feed the design equation of Section 4.3 predicts what step 4 converges to: async `send` at every intra-servicing-chain boundary where the combined stage times sit above the $2h$ floor of Section 4.3, and `fast_send` at every boundary between short sub-tasks below that floor. The operator does not need to trust that prediction to run the discipline; the measurement is the arbiter, and the arrival-law prior tells the operator where to expect it to land.

7.4 Separation of code construction from latency optimization

A consequence of Section 7.3 that deserves separate emphasis: in KASPAR-HFT *two* distinct topology decisions are not knowable at design time. The first is the *actor-to-thread mapping* — which actors share a thread by being placed in a common `Group`, and which run on their own thread. The second is the *per-boundary send-mode choice* — whether each pair of adjacent actors is joined by synchronous `fast_send` or asynchronous `send`. Neither decision is deducible from the source code alone. Both are functions of the target feed’s arrival statistics and the operator’s median-versus-tail tradeoff, both of which are properties of the deployment environment. The optimum can only be measured from the assembled system running against a representative feed.

Under receiver transparency this is a feature rather than a limitation. Actor code, message-type headers, and the full suite of unit and integration tests can be written and run before either the actor-to-thread mapping or the send-mode choice has been decided. Code construction and testing therefore proceed independently of the latency-optimization consideration — both decisions are deferred to the end of the build, informed by measurements that could not have been taken any earlier because the system did not yet exist to measure. Correctness is established first; latency is optimized against the specific feed the system will run on, second. Two developers building two actors on the two sides of a boundary do not need to agree on whether their actors will ultimately live on the same thread or on different threads, nor on whether their handoff will be synchronous or asynchronous; they need to agree only on the message type. The message-type agreement is checkable at compile time; the two topology decisions are scalar dials adjusted at deployment.

This decoupling of correctness-phase engineering from latency-phase engineering is the practical payoff of receiver transparency, and it is what makes KASPAR-HFT’s design cycle qualitatively different from frameworks in which either topology decision leaks into the receiver’s interface (Section 7.5). Design proceeds without a commitment to a topology; the commitment is made once,

near deployment, on the basis of measurements the code did not need to know about while it was being written.

7.5 Contrast with non-fungible frameworks

Most low-latency actor frameworks bake the sender’s choice of delivery mode into the receiver’s type. An Akka actor’s `tell` versus `ask`, a CAF actor’s `send` versus `request`, and an Erlang process’s `cast` versus `call` each expose different receiver-side surfaces, so flipping a boundary from sync to async is a small refactor rather than a one-line change. Under those frameworks the design flow of Section 7.3 is not tractable, and the operator’s incentive is therefore to lock in the send-mode choice early — reasoning locally — rather than to defer it to an end-of-build measurement pass. This structural incentive is a plausible contributing cause of the single-thread orthodoxy: frameworks that penalise deferral push designers toward the topology that is least sensitive to a wrong early guess, which is the collapsed single-thread servicing chain.

KASPAR-HFT’s joint mailbox API of `fast_send` and `send`, and the receiver-transparency property underneath, are the concrete machinery that make measurement-driven send-mode optimization practical. That is the framework property this paper’s design rule depends on operationally, and it is why we regard it as critical to system-level optimization on Hawkes-clustered feeds.

8 Conclusion

The paper’s two headline conclusions overturn the received wisdom in HFT system design. That wisdom holds that the entire hot path should live on a single-thread event loop, on the grounds that every thread hop adds real per-message latency; the community is overwhelmingly in the single-thread camp. We show on real CME MDP3 packet flow that both halves of the argument fail.

Conclusion 1: the thread-hop cost is a wash under real arrivals. The dominant term in end-to-end latency on a Hawkes-clustered feed is not the mailbox handoff — it is the cluster-wait tail. On the interim $T = 10\ \mu\text{s}$, $h = 3\ \mu\text{s}$ run the observed corpus-median p_{99} is $4\times$ the service floor and $\sim 13\times$ the hop cost, and Conjecture 2 predicts the same qualitative ratio across the running $h = 1.7\ \mu\text{s}$, $T \in \{2, 4, 8, 16, 32\}\ \mu\text{s}$ sweep. Adding a hop is $+1.7\ \mu\text{s}$ on the median; the tail it lets us compress is one to two orders of magnitude larger whenever T sits above the $2h$ floor. The single-thread objection — “every hop is a fixed tax that no benefit can overcome” — is empirically wrong on this class of feed.

Conclusion 2: two threads are strictly better than one. Splitting a service task into two half-length actors on separate threads joined by one $1.7\ \mu\text{s}$ mailbox hop is a Pareto improvement on every 30-minute window in the interim run and is expected to remain so across the $T \in \{4, 8, 16, 32\}\ \mu\text{s}$ rows of the running sweep. The $T = 2\ \mu\text{s}$ row sits below the $2h$ floor of Section 4.3 and is included explicitly to demonstrate the boundary, not to claim a win there. The general pattern extends to $N > 2$: an N -stage tandem contracts the tail by $N^{-1/2}$ or faster while paying $(N - 1)h$ on the median, up to the number of natural cut points in the decode chain (four or five on CME). The single-thread event loop is strictly dominated on Hawkes-clustered feeds above the $2h$ floor.

Why the received wisdom persists. We think the field arrived at the single-thread rule under a mental model of Poisson-like arrivals, where clustering does not exist and the tail sits near the service floor. On such a feed the received wisdom is correct — our own results give exactly that

answer under the Poisson null. What the field missed is that real exchange feeds have never been Poisson: near-critical branching ratios of $n \approx 0.9$ on CME futures have been documented for over a decade, and it is precisely those clusters that reverse the architectural recommendation. When your feed is bursty, thread-hop overhead is not a cost — it is the price of admission to a Pareto-better operating point.

What this changes in practice. Multi-actor pipelines are the correct architecture for HFT feeds specifically because those feeds are self-exciting. Framework designers should replace wait/notify async mailboxes with busy-poll dequeue on pinned isolated cores, drive the mailbox mutex uncontended, keep memory-pool discipline, and set N to the natural decomposition boundaries of the decode pipeline — 4–5 stages on CME. Strategy designers of both POV-style and race-sensitive algorithms benefit from the same N budget for the same reason: the tail is what matters, and the pipeline compresses it.

Retraction of a prior design context, and the corrected design rule. An earlier paper by the present author [31] did not itself argue for the single-thread servicing chain; it took the collapsed single-thread servicing chain as the widely accepted design context of the time and offered `fast_send` as the mechanism that makes actor composition inside it free of context-switch overhead. What the present paper retracts is the design context, not the mechanism: on Hawkes-clustered feeds the single-thread servicing chain should no longer be the unconditional default. The corrected position is that `fast_send` (synchronous, inline, sub-microsecond) and asynchronous `BQueue` handoffs (a real pipelined hop at a few microseconds) are *complementary* tools, and the choice between them is a per-boundary design decision that depends on the service time being chained and the arrival law of the feed the pipeline sits on. Designers should use `fast_send` and asynchronous sends judiciously, according to which is appropriate for the task at hand.

In shipping KASPAR-HFT the `fast_send` vs asynchronous-send choice at each actor boundary is normally left open until the end of the build, and is resolved by measuring the end-to-end median and tail latency of the assembled pipeline under the target feed. A convenient property of the KASPAR-HFT actor API is that the receiving actor’s `MESSAGE_HANDLER` is agnostic to how it was invoked: swapping a synchronous `fast_send` at a call site for an asynchronous `send` (or vice versa) is a one-line change on the sender side and requires no code change on the receiver side. The choice therefore lives entirely at each individual call site and can be flipped per boundary at end-of-build with minimal engineering cost. The paper’s design equation now supplies the arrival-law-aware default value of that end-of-build measurement: on Hawkes-clustered feeds, long tasks should be split into shorter actors joined by asynchronous sends as a matter of preference, on the presumption that splitting compresses the overall tail more than the added hop cost inflates the median. The rule presupposes that the task being split has a service time T comfortably above the $2h$ floor of Section 4.3; a task of order h or below cannot be usefully split (a $1\ \mu\text{s}$ task cannot be turned into two 500 ns actors when a single hop alone costs $1.7\ \mu\text{s}$). Short synchronous chains inside one actor’s servicing of one message therefore continue to use `fast_send` unchanged. The end-of-build measurement remains the authoritative arbiter of any specific boundary.

Main caveats. The corpus is a single instrument (NQ front-month, chan 318); ES and BTC replications are follow-on work. Constant service is a simplification; the true per-actor work has state-dependent variability we do not model here. The cache-pollution cost of spinning receivers on many cores is bounded but not measured. And the tail-ratio result is stated as a conjecture in Part I because a rigorous proof requires monotone-separable-network machinery adapted to Hawkes

batch inputs (Appendix A); the paper’s empirical Part II substitutes measurement for proof.

None of these caveats reverse the two conclusions on the 281-session corpus we tested.

9 Future work

- Multi-instrument replication (ES, BTC).
- Full waiting-time distribution of a Hawkes-driven series tandem (open theoretical problem, not closed by this paper).
- Cache-pollution empirical measurement per spinning-receiver stage count.
- Power-law-kernel Hawkes fit versus exponential.
- Overnight-session regime.
- Own-capture pcap validation of the network sub-stage.
- Load-conditioned hop cost $h(\rho)$. This paper calibrates $h = 1.7\mu\text{s}$ from the low-load wake-latency figure in Mayeski [31]; under sustained-load conditions inside a Hawkes cluster the receiver never sleeps and the effective h collapses toward the mutex + cache-line-transfer floor of a few hundred nanoseconds (Section 4.5). Measuring $h(\rho)$ directly and re-running the sweep against the load-conditioned h would sharpen the reported tail-win, which the current single- h sim understates.

10 Reproducibility

Code: `arrival_paper/qsim_grid.py` (per-window packet-Hawkes fit and N -stage tandem Lindley with Poisson null) and `arrival_paper/close_window_sim.py` (falsification of the span-scaled service term). Data: 281 message tapes at `/vast/home/vmayeski/out/arrival_paper/tapes/318/message`. Companion KASPAR-HFT simulator with two-queue Lindley upgrade is released alongside the paper.

References

- [1] Hawkes, A. G. (1971). Spectra of some self-exciting and mutually exciting point processes. *Biometrika* 58, 83–90.
- [2] Hawkes, A. G., Oakes, D. (1974). A cluster process representation of a self-exciting process. *Journal of Applied Probability* 11(3), 493–503.
- [3] Tanner, J. C. (1961). A derivation of the Borel distribution. *Biometrika* 48(1–2), 222–224.
- [4] Asmussen, S., Foss, S. (2003). Subexponential asymptotics of waiting-time and cycle-maximum distributions in single-server queues. In *Heavy-Tailed Distributions in Applied Probability*, Cambridge University Press.
- [5] Gao, X., Zhu, L. (2018). Functional central limit theorems for stationary Hawkes processes and application to infinite-server queues. *Queueing Systems* 90, 161–206.
- [6] Jackson, J. R. (1957). Networks of waiting lines. *Operations Research* 5, 518–521.

- [7] Burke, P. J. (1956). The output of a queuing system. *Operations Research* 4, 699–704.
- [8] Kelly, F. P. (1979). *Reversibility and Stochastic Networks*. Wiley.
- [9] Kleinrock, L. (1976). *Queueing Systems, Vol. II: Computer Applications*. Wiley.
- [10] Whitt, W. (1983). The queueing network analyzer. *Bell System Technical Journal* 62, 2779–2815.
- [11] Foss, S., Korshunov, D. (2000). Steady-state asymptotics for tandem, split-match and other feedforward queues with heavy-tailed service. *Queueing Systems* 35, 65–80.
- [12] Baccelli, F., Foss, S., Lelarge, M. (2005). Tail asymptotics for monotone-separable networks. *J. Appl. Probability* 42(3), 793–812.
- [13] Heindl, A. (2001). Decomposition of general tandem queueing networks with MMPP input. *Performance Evaluation* 44(1–4), 5–23.
- [14] Kim, Y. H., Chydziński, A. (2002). Connection-wise end-to-end performance analysis of queueing networks with MMPP inputs. *Performance Evaluation* 47(2–3), 137–156.
- [15] Dębicki, K., Mandjes, M. (2015). *Queues and Lévy Fluctuation Theory*. Springer.
- [16] Daw, A., Pender, J. (2018). Queues driven by Hawkes processes. *Stochastic Systems* 8(3), 192–229.
- [17] Koops, D. T., Boxma, O. J., Mandjes, M. R. H. (2018). Infinite-server queues with Hawkes arrivals. *Queueing Systems* 92, 27–52.
- [18] Chen, X., Blanchet, J. (2021). Perfect sampling of Hawkes processes and queues with Hawkes arrivals. *Stochastic Systems* 11(3), 264–283.
- [19] Sheldon, D., et al. (2023). Steady-state analysis and online learning for queues with Hawkes arrivals. arXiv 2311.02577.
- [20] Gao, X., Zhu, L. (2024). Single-server queues with state-dependent Hawkes arrivals. *Mathematics of Operations Research*.
- [21] Ozaki, T. (1979). Maximum likelihood estimation of Hawkes’ self-exciting point processes. *Ann. Inst. Statist. Math.* 31(1), 145–155.
- [22] Filimonov, V., Sornette, D. (2012). Quantifying reflexivity in financial markets: toward a prediction of flash crashes. *Physical Review E* 85, 056108.
- [23] Hardiman, S. J., Bercot, N., Bouchaud, J.-P. (2013). Critical reflexivity in financial markets: a Hawkes process analysis. arXiv 1302.1405.
- [24] Bacry, E., Mastromatteo, I., Muzy, J.-F. (2015). Hawkes processes in finance. *Market Microstructure and Liquidity* 1(1).
- [25] Byrd, D., Hybinette, M., Balch, T. H. (2020). ABIDES: Towards high-fidelity market simulation for AI research.
- [26] Frey, S., Li, K., Nagy, P., Sapora, S., Lu, C., Zohren, S., Foerster, J., Calinescu, A. (2023). JAX-LOB. arXiv 2308.13289.

- [27] Aquilina, M., Budish, E., O’Neill, P. (2022). Quantifying the high-frequency trading “arms race”. *Quarterly Journal of Economics* 137(1), 493–564.
- [28] Dean, J., Barroso, L. A. (2013). The tail at scale. *Communications of the ACM* 56(2), 74–80.
- [29] Zhao, K., Alizadeh, M., Pan, R., Ports, D. R. K., Perry, J., Yang, X., et al. (2023). Parsimon: scalable tail latency estimation for data center networks. NSDI 2023.
- [30] Mayeski, V. (in preparation). KASPAR-HFT: a shared-nothing actor framework for CME futures — `fast_send` hop-cost measurement.
- [31] Mayeski, V. (2025). `fast_send`: synchronous inline dispatch on the CME MDP3 hot path. M2 Tech technical report, KASPAR-HFT repository. https://github.com/vmayeski/kaspar-hft/blob/main/tech_reports/fast_send.pdf
- [32] Thompson, M. (2011). The Single Writer Principle. *Mechanical Sympathy*, 29 September 2011. <https://mechanical-sympathy.blogspot.com/2011/09/single-writer-principle.html>
- [33] Fowler, M. (2011). The LMAX Architecture. 12 July 2011. <https://martinfowler.com/articles/lmax.html>
- [34] LMAX Exchange (2011). Disruptor: high-performance alternative to bounded queues for exchanging data between concurrent threads. <https://lmax-exchange.github.io/disruptor/>
- [35] Rigtorp, E. (2020). Low latency tuning guide. <https://rigtorp.se/low-latency-guide/>
- [36] Chronicle Software (2023). Building fast trading engines: Chronicle’s approach to low-latency trading. <https://chronicle.software/insights/blogs/building-fast-trading-engines-chronicles-approach-to-low-latency-trading>
- [37] Real Logic (2016). Aeron: efficient reliable UDP unicast, UDP multicast, and IPC message transport. <https://aeron.io/>
- [38] Kwan, A., Wang, H., Luk, W. (2023). C++ design patterns for low-latency applications including high-frequency trading. arXiv 2309.04259.
- [39] Hu, W. (2024). The deterministic event-driven sequencer architecture: a competitive edge for high-frequency trading. Medium, October 2024. <https://medium.com/@hu.wenzhe124124/the-deterministic-event-driven-sequencer-architecture-a-competitive-edge-for-high-frequency>
- [40] Schmidt, D. C. (1995). Reactor: an object behavioral pattern for concurrent event demultiplexing and event handler dispatching. In *Pattern Languages of Program Design*, J. O. Coplien and D. C. Schmidt (eds.), Addison-Wesley.
- [41] Kegel, D. (1999). The C10K problem. <https://www.kegel.com/c10k.html>
- [42] Sysoev, I. (2004). nginx: a high-performance HTTP server and reverse proxy. <https://nginx.org/>
- [43] Dahl, R. (2009). Node.js: JavaScript runtime built on the V8 engine. Presented at JSConf EU, Berlin, November 2009. <https://nodejs.org/>
- [44] Welsh, M., Culler, D., Brewer, E. (2001). SEDA: an architecture for well-conditioned, scalable internet services. *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, 230–243.

- [45] Welsh, M. (2010). A retrospective on SEDA. <https://matt-welsh.blogspot.com/2010/07/retrospective-on-seda.html>
- [46] Kohler, E., Morris, R., Chen, B., Jannotti, J., Kaashoek, M. F. (2000). The Click modular router. *ACM Transactions on Computer Systems* 18(3), 263–297.
- [47] Murray, D. G., McSherry, F., Isaacs, R., Isard, M., Barham, P., Abadi, M. (2013). Naiad: a timely dataflow system. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 439–455.
- [48] Marty, M., de Kruijf, M., Adriaens, J., Alfeld, C., et al. (2019). Snap: a microkernel approach to host networking. *Proc. 27th ACM Symposium on Operating Systems Principles (SOSP)*, 399–413.
- [49] Kaffes, K., Chong, T., Humphries, J. T., Belay, A., Mazières, D., Kozyrakis, C. (2019). Shinjuku: preemptive scheduling for μ second-scale tail latency. *Proc. 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 345–359.
- [50] Ousterhout, A., Fried, J., Behrens, J., Belay, A., Balakrishnan, H. (2019). Shenango: achieving high CPU efficiency for latency-sensitive datacenter workloads. *Proc. 16th USENIX NSDI*, 361–378.
- [51] Fried, J., Ruan, Z., Ousterhout, A., Belay, A. (2020). Caladan: mitigating interference at microsecond timescales. *Proc. 14th USENIX OSDI*, 281–297.
- [52] Chen, H., Delimitrou, C., Kozyrakis, C. (2021). Perséphone: a microsecond-scale time-sharing scheduler for wide service-time distributions. *Proc. 28th ACM SOSP*, 653–668.
- [53] DPDK Project (2024). DPDK Programmer’s Guide: run-to-completion and pipeline modes. https://doc.dpdk.org/guides/prog_guide/
- [54] Zhang, X. (2016). Pipelined FAST protocol market-data decoder for high-frequency trading. Chinese patent CN105654383A.
- [55] Vinnikov, A. (2014). Batch latency analysis and phase transitions for a tandem of queues. arXiv 1403.2400.
- [56] David, T., Guerraoui, R., Trigonakis, V. (2013). Everything you always wanted to know about synchronization but were afraid to ask. *Proc. 24th ACM Symposium on Operating Systems Principles (SOSP)*, 33–48.
- [57] Belay, A., Prekas, G., Klimovic, A., Grossman, S., Kozyrakis, C., Bugnion, E. (2014). IX: a protected dataplane operating system for high throughput and low latency. *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 49–65.

A Structured proof sketch of Conjecture 2

This appendix records the intended shape of a rigorous proof of Conjecture 2, marked with the specific steps that require verification. It is not itself a proof; it is a scaffold for someone attempting one.

A.1 Setting and objective

Let the arrival process be the stationary Hawkes process of Section 2.2 with background rate μ , branching ratio $n \in (0, 1)$, and exponential kernel $\alpha e^{-\beta u}$ with $n = \alpha/\beta$. By the Hawkes–Oakes representation, arrivals are equivalent in distribution to a rate- μ Poisson immigrant process, each immigrant spawning an independent offspring cluster whose total size K is Borel(n)-distributed and whose lifetime L has tail $\Pr(L > u) \sim c \cdot u \cdot e^{-(1-n)\beta u}$ (Ogata) with the mean cluster duration $E[L] = \Theta(1/\beta(1-n))$.

Route this arrival process into an N -stage series tandem of single-server FIFO queues, each with deterministic service T/N and a deterministic hop delay h between consecutive stages. Let $W^{(N)}$ denote the steady-state end-to-end waiting time (sojourn minus service) of a typical message in stationary state.

Objective: show that in a suitable joint scaling ($n \rightarrow 1, \beta \rightarrow \infty$) with $\rho = \bar{\lambda}T$ bounded below 1,

$$\Pr(W^{(N)} > x) \sim \left(\frac{Nx}{T}\right)^{-1/2} \quad (x \rightarrow \infty), \quad (11)$$

and hence $\Pr(W^{(N)} > x)/\Pr(W^{(1)} > x) \rightarrow N^{-1/2}$.

A.2 Step-by-step scaffold

Step 1: Monotone-separable representation. Show that the N -stage tandem with deterministic service and deterministic hop delay is monotone-separable in the sense of Baccelli et al. [12, Def. 2.1]. This requires exhibiting the point-process input, an internal “epoch” representation, and monotonicity/separability functionals. For deterministic-service tandems this is essentially in Baccelli et al. [12, §3] but the batch-arrival extension needs explicit statement.

Gap 1: verify that the marked-point-process input (batch of size K at each immigrant time) fits the monotone-separable framework’s admissibility conditions; write out the “characteristic” functional χ of a single-batch input.

Step 2: Single-batch characteristic reduction. For a fixed batch size k , compute $\sigma_A(k)$, the total occupation time of the tandem by a single batch of size k arriving in an otherwise empty system. By Theorem 1, in the burst-saturation regime

$$\sigma_A(k) = T + (k-1)\frac{T}{N} + (N-1)h = \frac{kT}{N} + O(1). \quad (12)$$

This is the “ $\sigma_{\text{one-batch}}$ ” quantity that BFL2005 requires.

Gap 2: verify that Theorem 1’s burst-saturation assumption is uniformly satisfied for the batch sizes k that dominate the tail (large k); for small k the correction is bounded and does not affect the tail exponent.

Step 3: BFL2005 tail formula. Invoke Baccelli et al. [12, Thm. 3.1] to obtain

$$\Pr(W^{(N)} > x) \sim \frac{\bar{\lambda}}{1-\rho} \int_0^\infty \Pr(\sigma_A(K) > x + y\rho) dy \quad (13)$$

as $x \rightarrow \infty$, provided the characteristic $\sigma_A(K)$ is subexponential. The Borel distribution is intermediate regularly varying with tail exponent $1/2$, hence subexponential [4, Ch. 3].

Gap 3: verify that the subexponentiality class assumed by BFL2005 (usually $\mathcal{S}$ or $\mathcal{S}^*$) is satisfied by the Borel distribution’s $k^{-1/2}$ tail. This is likely direct but must be checked.

Step 4: Substituting the Borel tail. Combining Equations (12) and (13), and using $\Pr(K > k) \sim k^{-1/2}$ (Tanner 1961) plus $\sigma_A(k) \sim kT/N$:

$$\Pr(W^{(N)} > x) \sim \Pr\left(\frac{KT}{N} > x\right) \sim \left(\frac{Nx}{T}\right)^{-1/2}, \quad (14)$$

which is Equation (11), and dividing by the $N = 1$ analogue gives $N^{-1/2}$.

Gap 4: the integral in Equation (13) reduces to $\Pr(KT/N > x)$ up to a constant when σ_A is asymptotically linear in K . This reduction is standard for subexponential σ_A but should be written out explicitly.

Step 5: The near-critical scaling limit (the real obstacle). Steps 1–4 assume the batch approximation — clusters treated as instantaneous batches of Borel-distributed size. Real Hawkes clusters have duration $E[L] = \Theta(1/\beta(1-n))$, which diverges as $n \rightarrow 1$ at fixed β . The obstacle is: two large clusters that overlap in real time may jointly cause a tail event, breaking the single-big-batch principle.

The intended remedy is a joint scaling. Let $\beta \rightarrow \infty$ and $n \rightarrow 1$ such that $\beta(1-n) \rightarrow \infty$ (so cluster duration $\rightarrow 0$) while $\bar{\lambda} = \mu/(1-n)$ stays bounded. In this joint limit, cluster durations vanish relative to inter-cluster gaps and the batch approximation becomes exact.

Gap 5 (the real one): state and prove a joint-scaling limit theorem showing that the waiting-time distribution of the Hawkes-driven tandem converges to that of the batch-arrival tandem in the appropriate space and mode of convergence. This is the piece that would require a page or two of measure-theoretic care; Gao and Zhu [5] on Hawkes fluid limits and Koops et al. [17] on infinite-server Hawkes limits both develop similar joint scalings and are likely templates.

Gap 6: verify that the tail exponent is invariant under this scaling limit, i.e. the Borel-derived $k^{-1/2}$ tail of the batch survives to the limit.

Step 7: Hop-cost correction. The hop delay h enters $\sigma_A(k)$ additively as $(N-1)h$, so its contribution to Equation (13) is a translation of the integrand by a constant. Since the Borel tail is regularly varying with index $1/2$, this translation contributes $O((N-1)h/T)$ correction to the pre-factor and does not shift the exponent. Verify this by direct integration.

Gap 7: pin down the pre-factor as a function of (N, T, h) ; it should be continuous in h at $h = 0$.

Step 8: Corollary 3 under the batch approximation. Under a rate-matched Poisson null, the batch size collapses to $K \equiv 1$ almost surely. Then $\sigma_A(1) = T + (N-1)h$ is deterministic and the tandem is $M/D/1$ -in-series with light-tailed waiting time. The tail-ratio is exactly 1 in the asymptotic limit.

A.3 Summary of gaps

Gap	Difficulty	Comment
1	mechanical	Instantiate BFL2005 admissibility for batched input.
2	straightforward	Uniform burst-saturation for large k .
3	direct check	Verify Borel is in $\mathcal{S}$ (subexponential class).
4	standard	Standard subexponential integral reduction.
5	main obstacle	Joint (n, β) scaling-limit theorem.
6	follow-on	Tail-invariance under the joint scaling.
7	mechanical	Pre-factor evaluation.

Gaps 1–4 and 7 are together the “clean composition” scenario of the paper’s follow-up plan (4–6 person-weeks per the estimate in Section 9). Gap 5 is the piece that pushes the effort to 3–6 person-months. A rigorous version of Gap 5 alone would be the technical contribution of the follow-on paper.

B Reading list for a formal proof attempt

A prospective author of the rigorous proof of Conjecture 2 should have the following literature on hand. Page ranges are approximate and refer to the arXiv or journal version where marked.

Framework and top-level tail formula

- **Baccelli, Foss, Lelarge (2005)**, “Tail asymptotics for monotone-separable networks.” *J. Appl. Probab.* 42(3), 793–812; arXiv:math/0510117. Definitions of monotone-separable networks (§2), the characteristic σ_A construction (§2.3), and the main tail theorem (§3, Thm. 3.1) in the batch-arrival version. This is the load-bearing reference.
- **Foss, Korshunov (2000)**, “Steady-state asymptotics for tandem, split-match and other feedforward queues with heavy-tailed service.” *Queueing Systems* 35, 65–80. Original tandem heavy-tail result on which BFL2005 builds; the tail-in-service case, not tail-in-batch, but the intuition transfers.
- **Baccelli, Foss (2004)**, “Moments and tails in monotone-separable stochastic networks.” *Ann. Appl. Probab.* 14, 612–650. The earlier version of BFL2005; sometimes clearer on the subexponential-input tail formula.

Batch-arrival single-server tail

- **Asmussen, Foss (2003)**, *Applied Probability and Queues*, 2nd ed., Springer. Chapter 10 on heavy-tailed queues; the single-big-batch / single-big-service dichotomy for $M/G/1$ and $M^{[K]}/G/1$. Standard reference for the subexponential $\Pr(W > x) \sim \Pr(\rho/(1 - \rho) \cdot X_e > x)$ family of results.
- **Asmussen, Foss, Korshunov (2003)**, “Asymptotics for sums of random variables with local subexponential behaviour.” *J. Theor. Probab.* 16, 489–518. Closure properties of subexponential classes; needed for the intermediate-regularly-varying Borel distribution.
- **Foss, Korshunov, Zachary (2011)**, *An Introduction to Heavy-Tailed and Subexponential Distributions*, Springer. The packaged textbook. Chapters 3–5 for subexponentiality; Chapter 10 for tandem queue tails.

Hawkes process, cluster representation, near-critical scaling

- **Hawkes, Oakes (1974)**, “A cluster process representation of a self-exciting process.” *J. Appl. Probab.* 11(3), 493–503. The immigrant-plus-offspring representation and the identification of the offspring cluster with a Galton–Watson tree.
- **Tanner (1961)**, “A derivation of the Borel distribution.” *Biometrika* 48(1–2), 222–224. Origin of the Borel–Tanner distribution and its tail asymptotic.

- Brémaud, Massoulié (1996), “Stability of nonlinear Hawkes processes.” *Ann. Probab.* 24(3), 1563–1588. Stability and stationary distribution of Hawkes; needed for identifying the stationary regime the tandem operates in.
- Gao, Zhu (2018), “Functional central limit theorems for stationary Hawkes processes and application to infinite-server queues.” *Queueing Systems* 90, 161–206. Template for the joint scaling limit; how they handle the near-critical limit for the ∞ -server tandem is close to what Gap 5 needs.
- Koops, Boxma, Mandjes (2018), “Infinite-server queues with Hawkes arrivals.” *Queueing Systems* 92, 27–52. Another template; explicit joint scaling of (μ, α, β) for the fluid limit.

Ancillary — tandem heavy-traffic and Lévy inputs

- Dębicki, Mandjes (2015), *Queues and Lévy Fluctuation Theory*, Springer. Chapter 6 on tandem queues with Lévy input; relevant if the batch process is replaced by a compound-Poisson Lévy process at the fluid limit.
- Konstantopoulos, Richardson (2004), “Convergence limits for a tandem queueing network in heavy traffic” (or related tandem-heavy-traffic papers by these authors). Extends single-station heavy-traffic diffusion to feedforward networks; relevant if the fluid limit is replaced by a heavy-traffic diffusion limit.

Verification against empirical result

- Companion code: `arrival_paper/qsim_grid.py` (per-window packet-Hawkes fit and N -stage tandem Lindley recursion) and `arrival_paper/tandem_demo.py` (self-contained demo of the two-model comparison). The design equation $p_{99}(N) \approx p_{99}(1)/N^\gamma$ with the exponent γ read off from the corpus is the paper’s falsifiable prediction from Conjecture 2; a rigorous proof would recover $\gamma \geq 1/2$.